

Java Advanced/Mid-Level Practice Library — 500 Problems

20 domains × 25 problems. Mixed mid-to-advanced complexity. Each item is a self-contained hands-on exercise — implement it, don't just read about it. Suggested order: work top-to-bottom for a structured path, or jump to weak areas.

1. Core Java & Language Internals

1. Explain and demonstrate the difference between stack and heap memory with a code example.
2. Implement a program demonstrating pass-by-value semantics for objects in Java.
3. Write a program showing the difference between `==` and `.equals()` for String and wrapper classes.
4. Demonstrate String immutability and the String pool with code.
5. Implement a custom StringBuilder-like mutable string class.
6. Write code to demonstrate autoboxing/unboxing pitfalls (NullPointerException scenario).
7. Explain and demonstrate static vs instance initialization blocks with execution order.
8. Implement a program demonstrating the difference between final, finally, and finalize().
9. Write a program to demonstrate method overloading resolution rules with ambiguous calls.
10. Implement covariant return types in method overriding.
11. Demonstrate the diamond problem resolution using default methods in interfaces.
12. Write a program showing static vs dynamic (runtime) polymorphism.
13. Implement a class hierarchy demonstrating the Liskov Substitution Principle violation and fix.
14. Demonstrate varargs behavior with overloaded methods.
15. Write a program to show the effect of instance initializer blocks vs constructors.
16. Implement enum with abstract methods and constructor arguments.
17. Demonstrate try-with-resources with a custom AutoCloseable class.
18. Write a program showing array covariance and ArrayStoreException.
19. Implement a program demonstrating the Java Memory Model's happens-before relationship.
20. Demonstrate the use of transient and volatile keywords with examples.
21. Write a program comparing shallow copy vs deep copy (Cloneable vs copy constructor).
22. Implement a record class (Java 16+) and compare with a traditional POJO.
23. Demonstrate sealed classes and pattern matching for switch (Java 17+).
24. Write a program using text blocks and explain their use cases.

25. Implement a custom annotation with retention policy and process it via reflection.

2. OOP Design & SOLID Principles

1. Design a class hierarchy for a vehicle rental system following SOLID principles.
2. Refactor a God class into smaller classes applying Single Responsibility Principle.
3. Implement Open/Closed Principle using strategy pattern for a discount calculation system.
4. Demonstrate Liskov Substitution Principle violation using a Square-Rectangle example and fix it.
5. Implement Interface Segregation Principle by splitting a fat interface into role interfaces.
6. Demonstrate Dependency Inversion Principle using constructor injection without a framework.
7. Design an abstract class vs interface decision for a payment processing system.
8. Implement composition over inheritance for a Car-Engine relationship.
9. Design a plugin architecture using interfaces and dynamic loading.
10. Implement multiple interface inheritance with default method conflict resolution.
11. Design an immutable class with a builder pattern for a complex Order object.
12. Implement equals(), hashCode(), and compareTo() correctly for a custom Money class.
13. Design a class demonstrating encapsulation with validation in setters.
14. Implement a Visitor pattern to add new operations to a class hierarchy without modifying it.
15. Design a notification system using polymorphism for Email, SMS, and Push channels.
16. Implement an abstract factory for creating UI themes (Light/Dark).
17. Design a State pattern for an order lifecycle (Created, Paid, Shipped, Delivered).
18. Implement a Template Method pattern for a report generation pipeline.
19. Design a Decorator pattern to add features to a coffee ordering system.
20. Implement a Composite pattern for a file-system directory structure.
21. Design an Observer pattern for a stock price alert system.
22. Implement a Chain of Responsibility for a request approval workflow.
23. Design a class using the Null Object pattern to avoid null checks.
24. Implement a Fluent Builder API for constructing complex SQL-like queries.
25. Design a class hierarchy combining Strategy + Dependency Injection to favor composition over inheritance.

3. Collections Framework

1. Implement a custom LinkedList from scratch supporting add, remove, and iteration.
2. Compare ArrayList vs LinkedList performance for insertion/deletion at different positions.

3. Implement a thread-safe ArrayList alternative using CopyOnWriteArrayList and benchmark it.
4. Write a program to detect and remove duplicates from a List without using Set.
5. Implement a custom HashMap from scratch with put/get/remove and collision handling.
6. Demonstrate the difference between HashMap, LinkedHashMap, and TreeMap with ordering examples.
7. Implement an LRU cache using LinkedHashMap.
8. Write a program to find the intersection and union of two lists using the Collections API.
9. Implement a custom Comparator to sort a list of objects by multiple fields.
10. Demonstrate fail-fast vs fail-safe iterators with ConcurrentModificationException.
11. Implement a priority queue-based task scheduler.
12. Write a program using TreeSet to maintain a sorted, deduplicated collection of custom objects.
13. Implement a Trie (prefix tree) using Java collections for autocomplete.
14. Demonstrate the use of EnumMap and EnumSet for a state machine.
15. Implement a multi-value map (Map<K, List<V>>) helper utility class.
16. Write a program to group and partition a list of objects using Collectors.
17. Implement a bounded blocking queue using wait/notify without java.util.concurrent.
18. Demonstrate WeakHashMap usage for a cache that allows garbage collection of unused entries.
19. Implement an immutable List wrapper and demonstrate UnsupportedOperationException.
20. Write a program to merge two sorted lists into one sorted list efficiently.
21. Implement a custom Iterator for a binary tree (in-order traversal).
22. Demonstrate the performance difference between HashSet and TreeSet for large datasets.
23. Implement a frequency counter using Map and find the top-K frequent elements.
24. Write a program to implement a circular buffer using an array-backed structure.
25. Implement a graph adjacency-list representation using Map<Node, List<Node>> and perform BFS/DFS.

4. Generics & Type System

1. Implement a generic Stack<T> class with push, pop, and peek.
2. Write a generic method to find the maximum element in a list using bounded type parameters.
3. Implement a generic Pair<K, V> class with proper equals/hashCode.
4. Demonstrate wildcards (? extends T, ? super T) with a copy utility method (PECS principle).

5. Implement a generic Repository<T, ID> interface for a data access layer.
6. Write a program demonstrating type erasure and its implications with reflection.
7. Implement a generic Builder pattern using self-referential generics (CRTP-like).
8. Demonstrate bounded type parameters with multiple bounds (T extends Comparable<T> & Serializable).
9. Implement a generic BinaryTree<T extends Comparable<T>> with insert and traversal methods.
10. Write a program showing why generic arrays cannot be created directly, and workarounds.
11. Implement a generic Event<T> and EventListener<T> system for a pub-sub mechanism.
12. Demonstrate raw type usage pitfalls and unchecked warnings.
13. Implement a generic Cache<K, V> with a pluggable eviction strategy.
14. Write a generic method to swap elements in an array of any type.
15. Implement a generic Result<T, E> type (similar to Rust's Result) for error handling.
16. Demonstrate variance issues with generic collections (List<Object> vs List<String>).
17. Implement a generic Visitor<T> interface for processing heterogeneous collections.
18. Write a generic utility class for converting between DTOs and Entities using reflection.
19. Implement a type-safe heterogeneous container using Class<T> as a key.
20. Demonstrate the use of generic exceptions and their limitations in Java.
21. Implement a generic linked-list-based Queue<T> with enqueue/dequeue.
22. Write a program using generic recursive bounds for a fluent Comparator chain.
23. Implement a generic Matrix<T extends Number> class with basic arithmetic operations.
24. Demonstrate super type tokens for capturing generic type information at runtime.
25. Implement a generic ObjectPool<T> for reusable resource management.

5. Exception Handling

1. Design a custom exception hierarchy for a banking application (InsufficientFundsException, etc.).
2. Implement a global exception handling strategy using try-catch-finally with resource cleanup.
3. Write a program demonstrating checked vs unchecked exceptions and when to use each.
4. Implement exception chaining (cause) to preserve the original stack trace.
5. Demonstrate multi-catch blocks for handling multiple exception types.
6. Write a program showing the pitfalls of catching generic Exception or Throwable.
7. Implement a retry mechanism with exponential backoff for transient failures.
8. Design a validation framework that aggregates multiple exceptions before throwing.

9. Implement a circuit breaker pattern manually using exception counting.
10. Demonstrate suppressed exceptions in try-with-resources with multiple AutoCloseable resources.
11. Write a program showing exception handling best practices in a layered architecture (Controller-Service-Repository).
12. Implement a custom RuntimeException with error codes for API error responses.
13. Design an exception translation layer converting low-level exceptions to domain exceptions.
14. Write a program demonstrating finally block execution order with return statements.
15. Implement a logging aspect that captures and logs all uncaught exceptions.
16. Demonstrate the performance cost of exceptions vs error codes with a benchmark.
17. Implement a custom Thread.UncaughtExceptionHandler for background threads.
18. Design an exception-safe resource management pattern for JDBC connections.
19. Write a program showing how exceptions interact with try-with-resources and finally together.
20. Implement a validation pipeline that throws a composite exception with all field errors.
21. Demonstrate rethrowing exceptions with precise rethrow (Java 7+ improved type checking).
22. Design a global exception-handler component (framework-agnostic version of @ControllerAdvice).
23. Implement error handling for asynchronous CompletableFuture chains.
24. Write a program comparing Optional-based error handling vs exceptions.
25. Implement a dead-letter queue pattern for handling message processing failures.

6. Multithreading & Concurrency

1. Implement a producer-consumer problem using wait/notify.
2. Implement the same producer-consumer problem using BlockingQueue.
3. Write a program demonstrating race conditions and fix them using synchronized blocks.
4. Implement a thread-safe Singleton using double-checked locking.
5. Demonstrate a deadlock scenario with two threads and two locks, then fix it with lock ordering.
6. Implement a custom thread pool from scratch using a task queue and worker threads.
7. Write a program using ReentrantLock with tryLock for timeout-based locking.
8. Implement a read-write lock scenario using ReentrantReadWriteLock for a cache.
9. Demonstrate thread starvation and fix it with fair locking.
10. Implement a CountdownLatch-based scenario for coordinating multiple worker threads.
11. Write a program using CyclicBarrier for a multi-phase parallel computation.

12. Implement a Semaphore-based connection pool limiter.
13. Demonstrate the use of ThreadLocal for per-thread context (e.g., user session data).
14. Implement a producer-consumer pipeline with multiple stages using ExecutorService.
15. Write a program comparing synchronized method vs synchronized block performance.
16. Implement a custom atomic counter using AtomicInteger and compare with a synchronized counter.
17. Demonstrate the ABA problem in lock-free programming and how AtomicStampedReference solves it.
18. Implement a producer-consumer using Exchanger for two threads to swap data.
19. Write a program simulating the dining philosophers problem and resolve deadlock.
20. Implement a thread-safe lazy initialization holder pattern.
21. Demonstrate volatile vs synchronized for visibility guarantees with a flag-based example.
22. Implement a work-stealing task scheduler using ForkJoinPool.
23. Write a program to parallelize a recursive merge sort using ForkJoinPool.
24. Implement graceful shutdown of an ExecutorService handling in-flight tasks.
25. Design a rate limiter (token bucket algorithm) using threads and scheduling.

7. java.util.concurrent Deep Dive

1. Implement a task orchestration system using CompletableFuture.thenCompose and thenCombine.
2. Write a program demonstrating exception handling in CompletableFuture chains (exceptionally, handle).
3. Implement parallel API-call aggregation using CompletableFuture.allOf.
4. Demonstrate the difference between Future and CompletableFuture with a real-world example.
5. Implement a custom ThreadPoolExecutor with a custom RejectedExecutionHandler.
6. Write a program using ScheduledExecutorService for periodic task execution.
7. Implement a ConcurrentHashMap-based cache with computeIfAbsent for thread-safe lazy loading.
8. Demonstrate the difference between ConcurrentHashMap and Collections.synchronizedMap under load.
9. Implement a batching mechanism using BlockingQueue and a scheduled flush.
10. Write a program using CopyOnWriteArrayList for a thread-safe listener registry.
11. Implement a distributed-like task queue simulation using LinkedBlockingQueue with bounded capacity.
12. Demonstrate StampedLock for optimistic reading in a high-read scenario.
13. Implement a custom Executor that limits concurrent tasks per user/tenant.

14. Write a program using Phaser for dynamic multi-phase thread coordination.
15. Implement a timeout-aware task execution using Future.get(timeout).
16. Demonstrate CompletableFuture.supplyAsync with a custom Executor vs the common pool.
17. Implement an async pipeline for order processing (validate -> charge -> notify) using CompletableFuture.
18. Write a program comparing performance of ArrayBlockingQueue vs LinkedBlockingQueue.
19. Implement a debounce mechanism using ScheduledExecutorService.
20. Demonstrate proper shutdown and awaitTermination patterns for ExecutorService in a Spring Boot app.
21. Implement a fan-out/fan-in pattern using ExecutorService and CountDownLatch.
22. Write a program using ConcurrentSkipListMap for a thread-safe sorted map.
23. Implement virtual threads (Project Loom, Java 21) for a high-throughput I/O-bound service.
24. Demonstrate structured concurrency (Java 21 preview) for coordinated subtasks.
25. Implement a custom CompletionService-based parallel task processor with ordered result collection.

8. Java 8+ Streams & Functional Programming

1. Implement a data pipeline using Stream to filter, map, and collect a list of employees by department.
2. Write a program using Collectors.groupingBy with downstream collectors for multi-level grouping.
3. Implement a custom Collector for computing statistics (min, max, avg) in one pass.
4. Demonstrate the difference between map() and flatMap() with a nested list example.
5. Implement a lazy evaluation demonstration using Stream peek() and intermediate operations.
6. Write a program using parallel streams and measure performance gains/pitfalls.
7. Implement a Function composition chain using andThen and compose.
8. Demonstrate Optional chaining (map, filter, orElseGet) to avoid null checks.
9. Implement a custom functional interface with a default method for validation chaining.
10. Write a program using Stream.reduce() to implement a custom aggregation.
11. Implement a data transformation pipeline reading from a CSV and using Streams for processing.
12. Demonstrate Collectors.toMap with a merge function for handling duplicate keys.
13. Implement a Supplier-based lazy initialization for expensive objects.
14. Write a program using Predicate combinators (and, or, negate) for complex filtering logic.

15. Implement infinite stream generation with `Stream.iterate` and `limit` for a Fibonacci sequence.
16. Demonstrate the difference between `findFirst()` and `findAny()` in parallel streams.
17. Implement a stream-based word frequency counter from a large text file.
18. Write a program using `Collectors.teeing` (Java 12+) to compute two aggregates in one pass.
19. Implement method references (static, instance, constructor) replacing lambda expressions in a real example.
20. Demonstrate short-circuiting operations (`anyMatch`, `allMatch`, `noneMatch`) with early termination.
21. Implement a Stream-based JSON-like data transformer for nested objects.
22. Write a program comparing imperative vs functional style for a complex business report.
23. Implement a custom `Splitter` for a domain-specific collection.
24. Demonstrate switch expressions and pattern matching (Java 17+) replacing complex if-else chains.
25. Implement a functional-style validation framework using `Function<T, Optional<String>>` composition.

9. JVM Internals, GC & Memory Management

1. Write a program to demonstrate object creation and inspect memory using JVM flags (`-Xms`, `-Xmx`).
2. Implement a memory leak scenario (static collection holding references) and detect it.
3. Demonstrate the difference between Young Generation and Old Generation GC behavior.
4. Write a program to trigger and observe minor GC vs major/full GC using verbose GC logs.
5. Implement a scenario demonstrating strong, soft, weak, and phantom references.
6. Demonstrate the class loading process and a custom `ClassLoader` implementation.
7. Write a program to analyze heap dumps conceptually (explain steps using `jmap`/`VisualVM`/`Eclipse MAT`).
8. Implement a benchmark comparing G1GC vs Parallel GC for a memory-intensive application.
9. Demonstrate escape analysis and stack allocation optimization concepts with example code.
10. Write a program showing the impact of `finalize()` (deprecated) vs the Cleaner API for resource cleanup.
11. Implement a custom class loader that loads classes from a non-standard source (e.g., in-memory bytes).
12. Demonstrate the method area/metaspaces and an `OutOfMemoryError: Metaspaces` scenario.

13. Write a program to explain and simulate a `StackOverflowError` with recursion depth.
14. Implement JIT compilation observation using `-XX:+PrintCompilation` flag concepts.
15. Demonstrate object header layout and memory footprint differences between primitives and wrapper objects.
16. Write a program showing String interning behavior and memory implications.
17. Implement a scenario demonstrating the performance benefit of `StringBuilder` over String concatenation in loops.
18. Demonstrate thread stack size configuration and its effect using `-Xss`.
19. Write a program to profile CPU vs memory bottlenecks using Java Flight Recorder concepts.
20. Implement a custom in-memory cache with size-bound eviction to avoid `OutOfMemoryError`.
21. Demonstrate the difference between checked exceptions and Error classes (`OutOfMemoryError`, `StackOverflowError`) in terms of handling.
22. Write a program illustrating false sharing in multithreaded code and how `@Contended` helps.
23. Implement a benchmark using JMH (Java Microbenchmark Harness) for a hot method.
24. Demonstrate class initialization order (static blocks, static fields) across a class hierarchy.
25. Write a program explaining the difference between compile-time and runtime polymorphism from a JVM bytecode perspective.

10. File I/O & NIO.2

1. Implement a file copy utility using `java.nio.file.Files.copy` with progress tracking.
2. Write a program using `BufferedReader` vs `Files.lines()` to read a large file efficiently.
3. Implement a `WatchService`-based file system monitor for a directory.
4. Demonstrate memory-mapped file I/O using `FileChannel` and `MappedByteBuffer`.
5. Write a program to read/write CSV files without external libraries.
6. Implement a multi-threaded file processing pipeline using NIO.2 `AsynchronousFileChannel`.
7. Demonstrate the difference between `java.io.File` and `java.nio.file.Path` APIs.
8. Write a program to recursively walk a directory tree using `Files.walk()` and filter by criteria.
9. Implement a log file rotation mechanism based on file size.
10. Demonstrate serialization and deserialization of a custom object graph with versioning (`serialVersionUID`).
11. Write a program implementing a custom binary file format reader/writer using `DataInputStream/DataOutputStream`.
12. Implement a ZIP file creation and extraction utility using `java.util.zip`.

13. Demonstrate try-with-resources with multiple chained streams (FileInputStream -> BufferedInputStream -> ObjectInputStream).
14. Write a program to compare performance of NIO Channels/Buffers vs traditional I/O streams for large file processing.
15. Implement a simple key-value store persisted to disk using RandomAccessFile.
16. Demonstrate character encoding issues (UTF-8 vs ISO-8859-1) when reading/writing files.
17. Write a program implementing atomic file writes (write to temp file, then rename).
18. Implement a file-based producer-consumer pipeline (watch folder, process, archive).
19. Demonstrate the use of PathMatcher (glob patterns) for filtering files in a directory.
20. Write a program to merge multiple large sorted files into one (external merge sort).
21. Implement a checksum verification utility (MD5/SHA-256) for file integrity checks.
22. Demonstrate reading and writing properties files using java.util.Properties.
23. Implement a simple CSV-to-JSON converter without external libraries.
24. Write a program using FileLock to prevent concurrent access to a shared file.
25. Implement a streaming log parser that processes gigabyte-scale log files with constant memory.

11. Design Patterns (GoF)

1. Implement the Singleton pattern using an enum (the recommended thread-safe approach).
2. Implement the Factory Method pattern for creating different types of notification senders.
3. Implement the Abstract Factory pattern for creating families of related UI components.
4. Implement the Builder pattern for constructing a complex immutable configuration object.
5. Implement the Prototype pattern for cloning complex objects with deep-copy semantics.
6. Implement the Adapter pattern to integrate a legacy payment gateway with a new interface.
7. Implement the Bridge pattern separating abstraction from implementation for a messaging system.
8. Implement the Composite pattern for a menu system with nested sub-menus.
9. Implement the Decorator pattern for adding logging and caching to a service dynamically.
10. Implement the Facade pattern to simplify a complex subsystem (order processing with inventory, payment, shipping).
11. Implement the Flyweight pattern for managing a large number of similar objects efficiently (e.g., character glyphs).

12. Implement the Proxy pattern for lazy loading and access control to an expensive resource.
13. Implement the Chain of Responsibility pattern for a multi-level approval workflow.
14. Implement the Command pattern for an undo/redo text editor operation system.
15. Implement the Interpreter pattern for a simple expression evaluator.
16. Implement the Iterator pattern for a custom collection with multiple traversal strategies.
17. Implement the Mediator pattern for a chat room application decoupling participants.
18. Implement the Memento pattern for saving and restoring object state (undo functionality).
19. Implement the Observer pattern with a custom event-bus implementation.
20. Implement the State pattern for a traffic light system.
21. Implement the Strategy pattern for pluggable sorting/pricing algorithms.
22. Implement the Template Method pattern for a data import pipeline (CSV, JSON, XML).
23. Implement the Visitor pattern for a compiler-like AST node evaluation.
24. Implement a combination of Factory + Strategy patterns for a multi-tenant pricing engine.
25. Implement the Null Object pattern combined with Strategy for a default no-op handler.

12. Data Structures & Algorithms

1. Implement binary search and its variants (first/last occurrence) on a sorted array.
2. Implement merge sort and quicksort, and compare their performance on different input sizes.
3. Implement a balanced binary search tree (AVL or Red-Black tree basics) with insertion.
4. Implement Dijkstra's shortest path algorithm using a priority queue.
5. Implement a topological sort for a directed acyclic graph (course scheduling problem).
6. Implement a dynamic programming solution for the 0/1 Knapsack problem.
7. Implement the Longest Common Subsequence algorithm using DP.
8. Implement a sliding window algorithm to find the maximum sum subarray of size K.
9. Implement a two-pointer technique solution for the 3Sum problem.
10. Implement a graph cycle detection algorithm (Union-Find/Disjoint Set).
11. Implement a trie-based autocomplete system with prefix search.
12. Implement Kadane's algorithm for maximum subarray sum.
13. Implement an LRU cache with $O(1)$ get/put using HashMap + Doubly Linked List.
14. Implement a min-heap-based solution for the K closest points to origin problem.
15. Implement a backtracking solution for the N-Queens problem.
16. Implement a solution for detecting a cycle in a linked list (Floyd's algorithm).

17. Implement a solution to serialize and deserialize a binary tree.
18. Implement quickselect to find the Kth largest element in an array.
19. Implement a word ladder solution using BFS.
20. Implement a solution for the longest palindromic substring using dynamic programming.
21. Implement a solution for matrix rotation and spiral traversal.
22. Implement a solution for finding all permutations and combinations of a given set.
23. Implement a solution for the coin change problem (minimum coins) using DP.
24. Implement a rate-limiter algorithm (sliding window log) as a coding exercise.
25. Implement a consistent hashing algorithm for a distributed cache simulation.

13. Spring Core & Dependency Injection

1. Build a simple IoC container from scratch to understand how Spring's DI works internally.
2. Implement constructor, setter, and field injection in a Spring Boot application and compare trade-offs.
3. Implement a custom @Qualifier-based bean resolution for multiple implementations of an interface.
4. Demonstrate Spring bean lifecycle callbacks (@PostConstruct, @PreDestroy, InitializingBean, DisposableBean).
5. Implement a custom BeanPostProcessor to modify beans during initialization.
6. Demonstrate the difference between singleton, prototype, request, and session bean scopes.
7. Implement a custom FactoryBean for creating complex objects.
8. Demonstrate circular dependency resolution and when it fails in Spring.
9. Implement Spring profiles for dev/test/prod environment-specific configurations.
10. Demonstrate @ConditionalOnProperty and custom @Conditional annotations.
11. Implement a custom Spring Boot starter module for a reusable logging feature.
12. Demonstrate ApplicationContext vs BeanFactory differences with code.
13. Implement event-driven communication using ApplicationEventPublisher and @EventListener.
14. Demonstrate AOP (Aspect-Oriented Programming) implementing a logging aspect with @Around advice.
15. Implement a custom AOP aspect for method-level performance monitoring.
16. Demonstrate @Value and @ConfigurationProperties for externalized configuration binding.
17. Implement a custom Environment-based feature flag system.
18. Demonstrate the use of @Primary and @Order for bean resolution priority.
19. Implement a Spring Boot application with multiple DataSource configurations.
20. Demonstrate @Async method execution with a custom TaskExecutor.

21. Implement a custom validator using `@Validated` and a custom constraint annotation.
22. Demonstrate Spring Boot auto-configuration by writing a custom `@AutoConfiguration` class.
23. Implement a graceful shutdown hook for a Spring Boot application releasing external resources.
24. Demonstrate testing Spring context loading with `@SpringBootTest` and `@ContextConfiguration`.
25. Implement a modular multi-module Spring Boot project (api, service, repository layers) with proper dependency direction.

14. Spring Boot Auto-Configuration & Actuator

1. Build a Spring Boot REST API with layered architecture (Controller, Service, Repository, DTO).
2. Implement custom Actuator health indicators for external dependencies (DB, Kafka, third-party API).
3. Implement custom Actuator metrics using Micrometer for business KPIs.
4. Demonstrate Spring Boot DevTools for hot reload during development.
5. Implement a custom application.yml profile-based configuration for multiple environments.
6. Demonstrate Spring Boot's exception handling using `@ControllerAdvice` and `@ExceptionHandler`.
7. Implement request/response logging using a custom Filter or Interceptor.
8. Demonstrate CORS configuration for a Spring Boot REST API.
9. Implement API versioning strategies (URL, header, media type) in Spring Boot.
10. Demonstrate Spring Boot's embedded Tomcat configuration and switching to Jetty/Undertow.
11. Implement a custom banner and startup event listener for a Spring Boot app.
12. Demonstrate graceful degradation using Actuator readiness/liveness probes for Kubernetes.
13. Implement rate limiting for REST APIs using a custom interceptor (Bucket4j or manual token bucket).
14. Demonstrate Spring Boot configuration property validation with `@ConfigurationProperties` and JSR-303.
15. Implement a custom HandlerInterceptor for authentication token validation.
16. Demonstrate multipart file upload/download handling in Spring Boot REST APIs.
17. Implement OpenAPI/Swagger documentation for a Spring Boot REST API.
18. Demonstrate Spring Boot's `@ConditionalOnMissingBean` for creating overridable default beans.
19. Implement a scheduled job using `@Scheduled` with cron expressions and distributed locking consideration.

20. Demonstrate application startup optimization techniques (lazy initialization, component scanning reduction).
21. Implement a custom error response format (RFC 7807 Problem Details) across all endpoints.
22. Demonstrate Spring Boot Actuator's /env and /configprops endpoints security considerations.
23. Implement a health check aggregator combining multiple microservice health statuses.
24. Demonstrate Spring Boot testing slices (@WebMvcTest, @DataJpaTest, @JsonTest).
25. Implement a Spring Boot application packaged as a native image using GraalVM (conceptual walkthrough with code).

15. Spring Data JPA & Hibernate

1. Design an entity model with One-to-Many and Many-to-Many relationships and implement it with JPA.
2. Implement custom repository methods using @Query (JPQL and native SQL).
3. Demonstrate the N+1 select problem and fix it using JOIN FETCH or EntityGraph.
4. Implement optimistic locking using @Version for concurrent update handling.
5. Implement pessimistic locking for a critical inventory update scenario.
6. Demonstrate the difference between FetchType.LAZY and FetchType.EAGER with performance implications.
7. Implement Spring Data JPA Specifications for dynamic query building.
8. Demonstrate pagination and sorting using Pageable and Sort in a REST API.
9. Implement auditing (createdBy, createdAt, etc.) using @CreatedDate and @LastModifiedDate.
10. Demonstrate the first-level and second-level cache behavior in Hibernate.
11. Implement a custom converter (@Converter) for mapping a complex Java type to a database column.
12. Demonstrate cascading operations (CascadeType.ALL, orphanRemoval) with a parent-child entity example.
13. Implement a soft-delete mechanism using @Where and a boolean flag.
14. Demonstrate transaction propagation types (REQUIRED, REQUIRES_NEW, NESTED) with examples.
15. Implement a batch insert/update operation optimized for Hibernate performance.
16. Demonstrate the difference between merge() and persist() in JPA entity lifecycle.
17. Implement multi-tenancy using a discriminator column or schema-based approach in Spring Data JPA.
18. Demonstrate DTO projections (interface-based and class-based) to avoid overfetching.
19. Implement database migration using Flyway or Liquibase alongside Spring Data JPA.

20. Demonstrate handling of database deadlocks and retry logic in a Spring service layer.
21. Implement a composite primary key using @EmbeddedId or @IdClass.
22. Demonstrate the use of @Transactional read-only optimization for query-heavy services.
23. Implement full-text search integration using Hibernate Search or native SQL full-text queries.
24. Demonstrate entity state transitions (transient, persistent, detached, removed) with code examples.
25. Implement a complex reporting query using JPA Criteria API with dynamic filters and aggregations.

16. Spring Security & OAuth2/JWT

1. Implement basic authentication for a Spring Boot REST API.
2. Implement JWT-based stateless authentication with access and refresh tokens.
3. Demonstrate role-based access control (RBAC) using @PreAuthorize and @Secured.
4. Implement OAuth2 authorization code flow integrating with an external identity provider.
5. Demonstrate method-level security with custom SpEL expressions in @PreAuthorize.
6. Implement a custom UserDetailsService backed by a database.
7. Demonstrate password encoding using BCryptPasswordEncoder and migration strategies.
8. Implement CSRF protection configuration for a Spring MVC application.
9. Demonstrate CORS + Security configuration together for a SPA-backend integration.
10. Implement a custom AuthenticationProvider for multi-factor authentication.
11. Demonstrate securing actuator endpoints with role-based restrictions.
12. Implement API key-based authentication for service-to-service communication.
13. Demonstrate OAuth2 Resource Server configuration validating JWT tokens from an external issuer.
14. Implement a custom AccessDeniedHandler and AuthenticationEntryPoint for consistent error responses.
15. Demonstrate session management strategies (stateless vs session-based) trade-offs with code.
16. Implement a token blacklist/revocation mechanism for JWT logout functionality.
17. Demonstrate securing WebSocket connections in a Spring Boot application.
18. Implement fine-grained permission-based authorization (beyond roles) using a custom PermissionEvaluator.
19. Demonstrate integrating Spring Security with LDAP/Active Directory.
20. Implement rate limiting per authenticated user to prevent brute-force login attempts.
21. Demonstrate securing microservice-to-microservice calls using mutual TLS concepts with Spring Boot.

22. Implement audit logging for all authentication and authorization events.
23. Demonstrate handling of expired/invalid JWT tokens gracefully with proper error responses.
24. Implement single sign-on (SSO) integration conceptually using Spring Security SAML/OAuth2.
25. Demonstrate testing secured endpoints using `@WithMockUser` and `MockMvc`.

17. REST API Design & Best Practices

1. Design a RESTful API for an e-commerce order management system following resource-oriented principles.
2. Implement HATEOAS links in API responses using Spring HATEOAS.
3. Demonstrate proper HTTP status code usage across different API scenarios (200, 201, 204, 400, 404, 409, 422, 500).
4. Implement idempotency for POST requests using idempotency keys.
5. Design and implement API pagination, filtering, and sorting query parameter conventions.
6. Implement content negotiation supporting both JSON and XML responses.
7. Demonstrate ETags and conditional requests (If-None-Match) for caching optimization.
8. Implement API rate limiting with proper 429 Too Many Requests responses and Retry-After headers.
9. Design a versioning strategy for breaking API changes and implement backward compatibility.
10. Implement a bulk operations endpoint (batch create/update) with partial failure handling.
11. Demonstrate proper error response design following RFC 7807 (Problem Details for HTTP APIs).
12. Implement long-running operation handling using async patterns (202 Accepted + polling/webhook).
13. Design and implement a search API with complex filtering using query parameter parsing.
14. Implement request validation using Bean Validation with custom error message formatting.
15. Demonstrate API documentation generation using SpringDoc OpenAPI with examples and schemas.
16. Implement a file upload API with proper multipart handling and size/type validation.
17. Design an API gateway routing configuration for multiple backend microservices.
18. Implement request/response compression (GZIP) for a Spring Boot API.
19. Demonstrate proper use of PATCH vs PUT for partial vs full resource updates.
20. Implement API throttling per client tier (free vs premium) using interceptors.

21. Design a webhook delivery system with retry and signature verification.
22. Implement a GraphQL API alongside REST for flexible client queries (Spring GraphQL).
23. Demonstrate an API deprecation strategy with sunset headers and migration guides.
24. Implement a health-check and readiness API contract for load balancer integration.
25. Design and implement an API for streaming large datasets using Server-Sent Events (SSE).

18. Microservices Patterns

1. Implement the Saga pattern (choreography-based) for a distributed order-payment-inventory transaction.
2. Implement the Saga pattern (orchestration-based) using a central coordinator service.
3. Implement the CQRS pattern separating read and write models for a product catalog service.
4. Implement Event Sourcing for an account balance service with event replay.
5. Implement the Circuit Breaker pattern using Resilience4j for a downstream service call.
6. Implement the Bulkhead pattern to isolate thread pools for different downstream dependencies.
7. Implement the Retry pattern with exponential backoff and jitter using Resilience4j.
8. Implement the API Gateway pattern using Spring Cloud Gateway with routing and filters.
9. Implement service discovery using Eureka for dynamic microservice registration.
10. Implement client-side load balancing using Spring Cloud LoadBalancer.
11. Implement the Strangler Fig pattern for migrating a monolith to microservices incrementally.
12. Implement distributed configuration management using Spring Cloud Config Server.
13. Implement the Database-per-Service pattern and handle cross-service data consistency.
14. Implement distributed tracing across microservices using Micrometer Tracing and Zipkin.
15. Implement the Outbox pattern to ensure reliable event publishing alongside database transactions.
16. Implement the Anti-Corruption Layer pattern when integrating with a legacy system.
17. Implement a Backend-for-Frontend (BFF) service aggregating multiple microservice calls.
18. Implement distributed caching using Redis across multiple microservice instances.
19. Implement idempotent message consumers to handle duplicate message delivery.
20. Implement the Sidecar pattern concept for cross-cutting concerns (logging, metrics) in a containerized setup.

21. Implement graceful service degradation when a dependent microservice is unavailable (fallback methods).
22. Implement distributed locking using Redis (Redisson) for a critical section across service instances.
23. Implement contract testing between microservices using Spring Cloud Contract.
24. Implement correlation ID propagation across microservice calls for end-to-end request tracing.
25. Implement a health-check aggregation dashboard combining multiple microservices' status (custom Actuator extension).

19. Kafka & Event-Driven Architecture

1. Implement a Kafka producer and consumer for an order-events topic using Spring Kafka.
2. Implement a custom Kafka message key strategy for partition-based ordering guarantees.
3. Demonstrate consumer group rebalancing behavior with multiple consumer instances.
4. Implement exactly-once semantics using Kafka transactions in a Spring Boot producer.
5. Implement a dead-letter topic (DLT) strategy for handling failed message processing.
6. Demonstrate manual offset commit strategies (sync vs async) and their trade-offs.
7. Implement a Kafka Streams application for real-time aggregation (e.g., order count per minute).
8. Demonstrate schema evolution using Avro and Confluent Schema Registry.
9. Implement idempotent producers to avoid duplicate message delivery.
10. Demonstrate the difference between at-most-once, at-least-once, and exactly-once delivery semantics with code.
11. Implement a saga choreography using Kafka events between order, payment, and inventory services.
12. Demonstrate consumer lag monitoring and alerting concepts for a Kafka-based pipeline.
13. Implement a retry topic pattern with delayed reprocessing for transient failures.
14. Demonstrate Kafka partitioning strategy design for a multi-tenant event stream.
15. Implement a Kafka Connect source/sink connector configuration for database CDC (conceptual + config).
16. Demonstrate testing Kafka producers/consumers using EmbeddedKafka in Spring Boot tests.
17. Implement message deduplication logic at the consumer side using a processed-message store.
18. Demonstrate batch consumption and processing for high-throughput Kafka consumers.

19. Implement a custom Kafka serializer/deserializer for a complex domain object.
20. Demonstrate the Outbox pattern implementation, publishing database changes to Kafka reliably.
21. Implement a windowed aggregation (tumbling/sliding window) using Kafka Streams.
22. Demonstrate a consumer rebalance listener implementation for graceful partition handoff.
23. Implement multi-topic consumption with different processing logic per topic in one listener container.
24. Demonstrate Kafka security configuration (SASL/SSL) conceptually with Spring Boot properties.
25. Implement an event-driven choreographed saga with a compensating transaction on failure (rollback simulation).

20. Testing, Reactive Programming & Cloud-Native Java

1. Write comprehensive unit tests for a service layer using JUnit 5 and Mockito.
2. Implement integration tests using `@SpringBootTest` with an embedded/test database (H2 or TestContainers).
3. Demonstrate TestContainers usage for spinning up a real PostgreSQL/Kafka instance during tests.
4. Implement parameterized tests using JUnit 5's `@ParameterizedTest` for multiple input scenarios.
5. Demonstrate mocking static methods and final classes using Mockito's inline mock maker.
6. Implement contract tests for a REST API using Spring Cloud Contract or Pact.
7. Demonstrate test coverage analysis and identify untested edge cases in a payment service.
8. Implement a reactive REST API using Spring WebFlux with Mono and Flux.
9. Demonstrate backpressure handling in a reactive stream using Project Reactor.
10. Implement a reactive database access layer using R2DBC.
11. Demonstrate error handling strategies in reactive streams (`onErrorResume`, `onErrorReturn`, `retry`).
12. Implement a reactive WebClient-based service aggregating multiple downstream API calls concurrently.
13. Demonstrate testing reactive streams using StepVerifier.
14. Implement a Dockerfile and multi-stage build for a Spring Boot application.
15. Demonstrate writing a docker-compose.yml for a Spring Boot app with PostgreSQL, Redis, and Kafka.
16. Implement Kubernetes deployment and service YAML for a Spring Boot microservice with health probes.
17. Demonstrate horizontal pod autoscaling configuration concepts for a Java microservice under load.

18. Implement a CI/CD pipeline configuration (GitHub Actions or Jenkins) for build, test, and deploy stages.
 19. Demonstrate structured logging (JSON logs) integration with ELK/Loki for a Spring Boot app.
 20. Implement distributed tracing visualization setup conceptually using OpenTelemetry and Jaeger.
 21. Demonstrate blue-green and canary deployment strategies conceptually for a Java microservice.
 22. Implement mutation testing using PIT to evaluate test suite quality.
 23. Demonstrate performance/load testing a REST API using JMeter or Gatling scenarios.
 24. Implement a chaos engineering experiment (simulated latency/failure injection) for a microservice dependency.
 25. Demonstrate a complete CI pipeline running unit, integration, and contract tests with quality gates (SonarQube).
-

How to use this

- **Sequencing:** Sections 1–9 are core language/JVM depth (good for interview theory + fundamentals). Sections 10–12 round out I/O, patterns, and algorithms. Sections 13–20 are Spring/microservices/Kafka — directly relevant to your Java Lead and Connected Health platform work.
- **Portfolio tie-in:** Many of these map cleanly onto your existing PayFlow, OrderFlow, and RetireEdge360-style projects — implement problems from Sections 13–19 directly inside OrderFlow's Saga/Kafka setup to deepen that project rather than starting fresh each time.
- **Pacing:** At 5 problems/day this is a 100-day track; at 10/day, 50 days.